Mongo DB



Francisco Javier Báñez Arés

What is Mongo?

Universidad Francisco de Vitoria
UFV Madrid

MongoDB is a **source-available**, **cross-platform**, **document-oriented** database program. It is classified as a **NoSQL** db, which means it does not follow the traditional table-based relational database structure. Instead, it uses JSON-like documents with **optional schemas**, making it **incredibly versatile** and flexible for handling data.

MongoDB is a **distributed** database at its core (built with scalability, high availability, and geographic distribution in mind). It's **capable of supporting vast amounts of data across many servers**, making it ideal for businesses with substantial, dynamic datasets.

Ease of use is one of the core strengths of MongoDB. Developers find it intuitive due to its u**se of the BSON format**, which is a **binary representation** of **JSON-like documents.** This allows for easy data representation in **hierarchical**, **nested** data structures, which closely **align** with **object-oriented programming paradigms.**

The **robust documentation** and **active community support** are also significant factors that contribute to MongoDB's popularity. The official documentation is comprehensive, providing clear explanations and practical examples. There are also many online forums and resources where users can ask questions and share their experiences.

MongoDB University provides free online training courses using programming languages like Node.js, Java, Python, etc. and use cases, enabling developers to learn MongoDB within the context of their preferred programming language or specific application needs. Great way to deepen your existing knowledge. https://university.mongodb.com/ is the official site.

MongoDB Atlas is a fully-managed cloud database service provided by MongoDB. It allows users to deploy, operate, and scale their MongoDB databases on various cloud platforms like AWS, Google Cloud, and Azure. With Atlas, you can manage your databases without the operational overhead of self-managing the infrastructure.

Server Side Public License (SSPL)

Universidad Francisco de Vitoria

UFV Madrid

Software license introduced by MongoDB, Inc. It was crafted in response to cloud providers offering MongoDB as a service without contributing back to MongoDB's open-source ecosystem. SSPL aims to ensure that companies that provide MongoDB as a service must release the software they use to offer such services as open source.

While the SSPL is similar to an open-source license in that it allows the source code to be available, it contains additional conditions regarding the use and distribution of the software, particularly in a service-based context.

"Source-available" refers to the fact that Mongo's source code is publicly available for review and inspection.

The Open Source Initiative (OSI) has not approved SSPL as an open-source license. As a result of this and other considerations, some organizations treat SSPL-licensed software differently from software under traditional open-source licenses.

Let's say you have a **web application** where MongoDB, licensed under SSPL, is used as the **backend** database. You host this application on your servers or any cloud provider like AWS, Google Cloud, etc., and users access it. You are not required to release the source code of your web application under SSPL because **you are using MongoDB as a tool** and not providing it as a service.

imagine you start a new **cloud service** called "MyDBCloud" users can create, manage, and access Mongo databases. This is a direct service using MongoDB. Implication: SSPL would require you to release the entire software stack of "MyDBCloud" (including all management tools, UIs, automation scripts, etc.) under the SSPL.

Optional Schema validator

```
db.createCollection("students", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: [ "name", "year", "major" ],
      properties: {
        name: {
          bsonType: "string",
          description: "must be a string and is required"
        },
        year: {
          bsonType: "int",
          minimum: 1,
          maximum: 4,
          description: "an integer in [ 1, 4 ] and is required"
        },
        major: {
          enum: [ "CS", "EE", "ME", "CE" ],
          description: "one of the enum values, required"}
}}}
})
```

If any of these conditions are not met, MongoDB will not allow the insert operation

When you insert a document into the "students" collection, MongoDB **checks** that:

1. The document is an **object**.
2. The object **contains** the "name", "year", and "major" fields.
3. The "name" field is a **string**.
4. The "year" field is an **integer** between 1 and 4.
5. The "major" field is one of the **allowed values** ("CS", "EE", "ME", or "CE").

This schema is **optional**. **If you do not specify a validator when creating a collection, you can insert documents with any structure into the collection.**

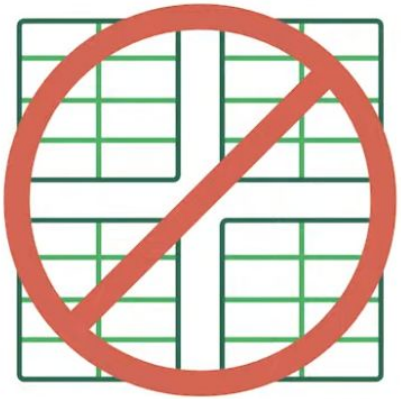
```
db.createCollection("students")
db.students.insert({name: "Alice", year: 3})
db.students.insert({name: "Bob", major: "CS"})
```

Both insert operations are allowed, even though the documents have different structures.

Document Oriented NoSQL

Universidad Francisco de Vitoria
UFV Madrid

MongoDB is a NoSQL database that focuses on document-oriented storage. Unlike traditional relational databases that store data in tables, MongoDB stores data in **BSON documents**, which is a **binary representation of JSON documents**. This gives MongoDB a lot of flexibility in terms of the structure of the data being stored.



The term 'NoSQL' is used to describe MongoDB not because it can't handle structured data (it can), but because **it doesn't use the traditional row-and-column format of SQL databases.** Despite this, MongoDB provides a comprehensive query language and strong consistency, ensuring that it can manage and organize data in a structured and reliable manner, like any good database should.

While data is represented as **key-value pairs** within a **BSON** document, it's important to note that the value can be a **complex type** such as an **array** or another **document**, allowing for rich data structures within a single document. This makes MongoDB well suited to handle **complex**, **hierarchical**, and **evolving data models**.

JS and Node.js pair well with MongoDB due to the JSON-like nature of BSON, Also MongoDB provides **drivers for a wide range of programming languages.** These drivers handle the conversion between the native types of the programming language and BSON, making MongoDB a versatile choice for many different software projects.

k-v yes, but Mongo can play with the value (document)

Universidad Francisco de Vitoria
UFV Madrid

A document must have an **unique _id field**, so could be stated that it is a Key Value store, where the key is the id (for sharding and so on) and the value is the document itself, but while this is kind of true it doesn't reflect the bigger power of Mongo versus pure **Key Value NoSQLs** such as **Redis** or **AWS DynamoDB**

While it's true that NoSQL databases are often referred to as "key-value" stores, this characterization is a bit of an oversimplification and doesn't fully capture the variety of data models supported by NoSQL databases.

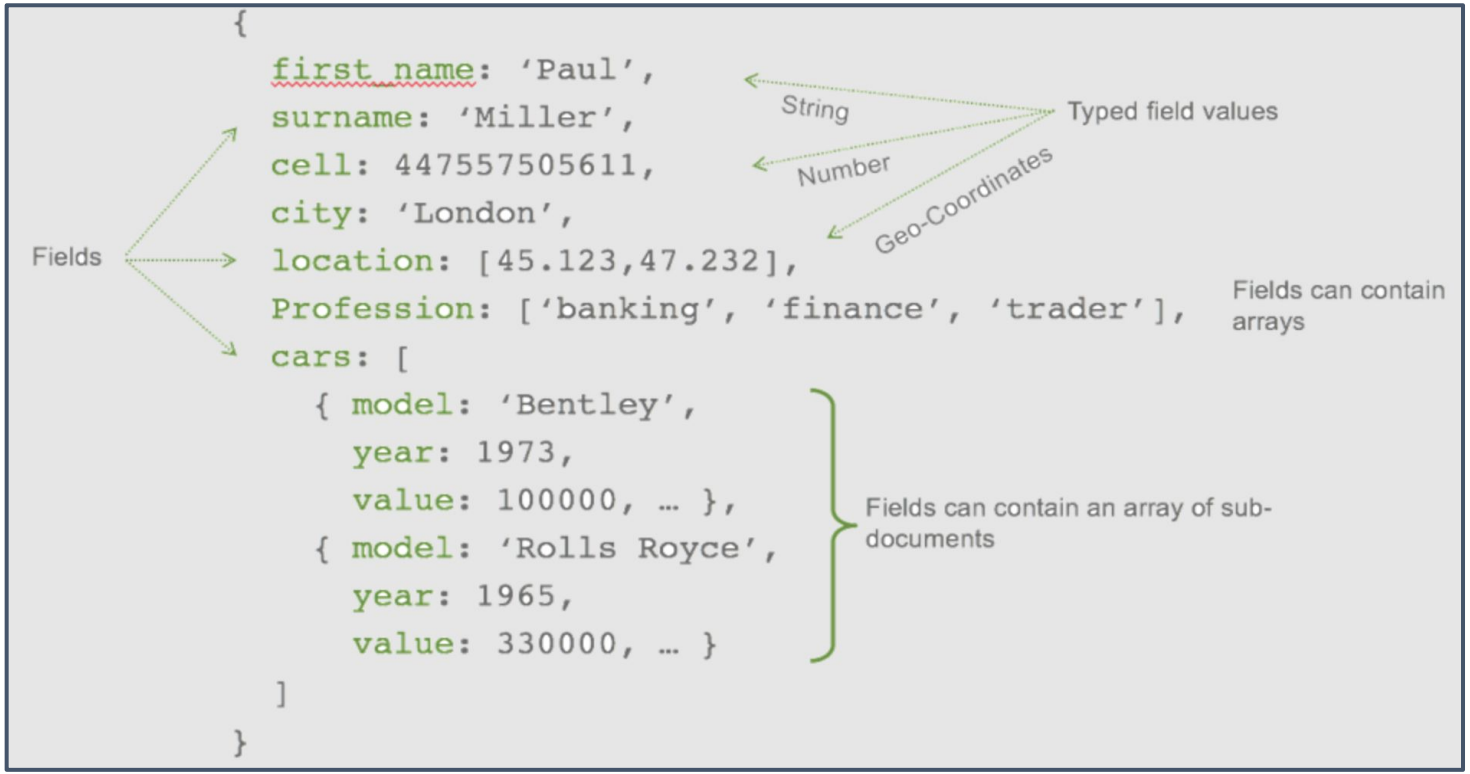
In a **key-value store**, each key is associated with a single **value**, and the database system treats that value as a **single** opaque blob -- it doesn't understand the internal structure of the value. **You can't query or index the contents of the value; you can only retrieve or replace it using its key.**

In contrast, **MongoDB understands the internal structure of its values** (which are BSON documents). This allows you to **query documents based** on their **contents**, create **indexes on any field** of a document, and **update individual fields within documents without replacing the entire document**. This is a richer and more flexible data model than what you get with a simple key-value store.

Even though MongoDB documents do have a unique identifier field (the **"_id" field,** which often contains an **ObjectId**), that doesn't make MongoDB a key-value store in the traditional sense. The "_id" field is just one of many fields within a document, and you can query and index on any field, not just the "_id" field.

So while it's not wrong to think of MongoDB as storing pairs of keys (like the "_id") and values (the rest of the document), calling it a "key-value" database might lead to misunderstandings, because that term is usually associated with a simpler data model than what MongoDB actually provides. Redis would be that simpler for example.

Welcome to the Document



Imagine it as a JSON, set of fields and values, one of the most common serialization human readable along with YAML

It will really be a BSON but don't worry as we will have our Driver making the format conversions

Field - a unique **identifier** for a datapoint.
Value - **data related** to a given identifier, can be a document

Documents and BSON

Universidad Francisco de Vitoria
UFV Madrid

This doc represents a person named John Doe, and contains an array of addresses. The **_id** field is a special field required in every Mongo document. This field holds the document's unique identifier.

```
{
 _id: ObjectId("5099803df3f4948bd2f98391"),
 name: { first: "John", last: "Doe" },
 birthdate: new Date('1990-01-01T00:00:00Z'),
 addresses: [
   { street: "123 Sesame St", city: "Anytown", cc: "USA" },
   { street: "123 Avenue Q", city: "New York", cc: "USA" }
 ]}
```

A document is a data structure composed of field and value pairs, much like JSON objects but Mongo documents use the Binary JSON (BSON) data format, which extends the JSON model to provide additional data types, ordered fields, and to be efficient for encoding and decoding within different languages.

BSON supports more data types than standard JSON, including **Date** and **ObjectId** used in this example.
These additional data types enable BSON to represent **rich** and **complex data structures** within **MongoDB documents**.

When working with MongoDB in your programming language of choice, you typically don't need to worry about the BSON format. MongoDB drivers for different languages (like Node.js, Python, Java, etc.) handle the **translation between BSON and the native data types in your language.**

For example, a **JavaScript** object will be automatically converted to BSON when you insert it into the database, and BSON documents will be automatically converted back to JavaScript objects when you query them. This translation is done in an efficient manner, making it easy and seamless to work with MongoDB documents.

JSON and BSON

Universidad Francisco de Vitoria
UFV Madrid

While BSON is based in JSON, it is **Binary** which make it better for computation, sending over the network, this serialization is much more efficient than **JSON that is UTF-8 known for being not efficient.**

Stored BSON vs. Viewed JSON

BSON also add some Data Types complementing the simplistic JSON

	JSON	BSON
Encoding	UTF-8 String	Binary
Data Support	String, Boolean, Number, Array, Object, null	String, Boolean, Number (Integer, Float, Long, Decimal128...), Array, null, Date, BinData
Readability	Human and Machine	Machine Only

https://www.mongodb.com/json-and-bson?_ga=2.135639671.717727541.1665716180-1126270963.1664671842

BSON

Universidad Francisco de Vitoria
UFV Madrid

BSON, or **Binary JSON**, is not just a play on words or a twist on JSON, it's a thoughtful and meticulously **crafted answer** to the **constraints** of JSON. It takes **JSON** - that simple, human-readable format we all know and love - and infuses it with **superpowers**.

```
{
  "name": "Juliet",
  "age": 27,
  "hobby": "gazing at the stars"
}
```

You can add dates, binary types, and other complex data that JSON can't handle. Now, your love letter becomes something more vibrant and full of life.

This is the wonderful world of BSON. It not only allows your love letter to be **richer** and more **complex**, but it also ensures that it's **stored in the most efficient way** possible, so you can retrieve and relive it whenever your heart desires.

```
{
  "name": "Juliet",
  "age": 27,
  "hobby": "gazing at the stars",
  "birthdate": ISODate("1995-07-20"),
  "photo": BinData("..."),
  "previous_letters": [
    ObjectId("507f1f77bcf86cd799439011"),
    ObjectId("507f1f77bcf86cd799439012")
  ]
}
```

JS / Node.js: MongoDB Node.js Driver
Python: PyMongo
Java: MongoDB Java Driver
C# / .NET: MongoDB .NET Driver
Ruby: MongoDB Ruby Driver
PHP: MongoDB PHP Driver
Go: MongoDB Go Driver
Rust: MongoDB Rust Driver
C++: MongoDB C++ Driver
Swift: MongoDB Swift Driver
Scala: MongoDB Scala Driver

But the best part of it all is that you don't have to worry about these technical details. Your **MongoDB driver does all the heavy lifting, converting your objects into BSON and back**, allowing you to focus on what really matters: crafting and telling beautiful love letters.

Collection, think of tables

Universidad Francisco de Vitoria
UFV Madrid

The idea of “table” in a RDBMS, but here **the idea is that the data is related and probably documents in a collection will have the same fields but not necessarily**

```
db.users.insert({
  name: "Alice",
  age: 30,
  email: "alice@email.com"
});
```

if the users collection **doesn't exist**, MongoDB will **create** it and then insert the document.

```
// Retrieve all users
db.users.find();

// Retrieve users with a specific condition
//(age greater than 25)
db.users.find({ age: { $gt: 25 } });
```

```
db.users.insert({
  username: "bob123",
  date_joined: new Date(),
  membership_status: "premium"
});
```

we've inserted a document with entirely different fields than the first. Both documents reside peacefully in the same users collection

Collections in MongoDB are **schemaless** don't require a predefined schema. You can insert documents with different fields into the same collection. Over time, as the needs of your application change, the structure of documents within a collection can evolve without needing major schema alterations.

Given the schema-less nature, collections offer a lot of flexibility. If your application logic requires a new field or drops an existing one, you can adjust your data model without the administrative overhead you might encounter in a traditional RDBMS.

Replica Set, giving mongodb high availability

Universidad Francisco de Vitoria

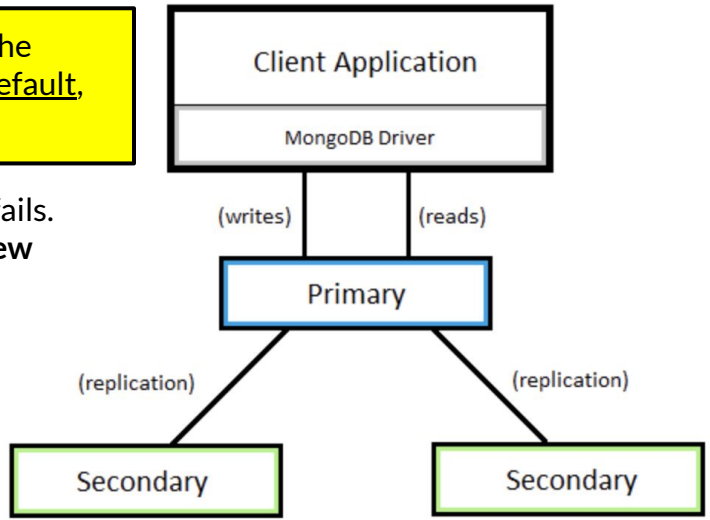
UFV Madrid

A MongoDB replica set is a **group of mongod instances that maintain the same data set.** Replica sets **provide redundancy** and **high availability**, and are the basis for all production deployments.

Replica sets represent the heart of MongoDB's approach to **distributing data** across multiple machines or geographic locations, while ensuring data **consistency** and **accessibility**.

The primary purpose of a replica set is to offer **fault tolerance** and **prevent data loss** through the **replication of data** across multiple servers or nodes, hence the term "replica set". This redundant architecture ensures that in the event of a failure—be it a system crash, network partition, or hardware fault—the database can **continue operating without loss of data or accessibility.**

In a MongoDB replica set, one node is designated as the "**primary**" node, while the others are "**secondary**" **nodes**. All **write operations** go to the primary node by default, and are then **asynchronously replicated** to the secondary nodes.



This design allows the system to continue functioning even if the primary node fails. In such an event, the remaining secondary nodes hold an **election** to **choose** a **new primary** node from among themselves, ensuring continuity of operations.

Imagine a choir where multiple singers are singing the same melody. If one singer were to suddenly lose their voice (analogous to a server failure), the song continues without interruption because the other singers (servers) are maintaining the melody (data).

Scalability, Availability, Consistency. AP

Universidad Francisco de Vitoria
UFV Madrid

In addition to fault tolerance, replica sets offer other advantages like **read scalability** (by distributing read operations across multiple nodes) and **data locality** (by **placing data closer to where it's needed**).

MongoDB replica sets are a powerful feature that can provide robustness, **availability**, and resilience in the face of hardware failures or network partitions, all while maintaining the **integrity** and **consistency** of your data.

Remember, the beauty of replica sets doesn't just lie in their ability to keep data safe and available—it's also in their capability to scale with your needs, grow with your applications, and evolve with the ever-changing landscape of data management.

MongoDB replica sets are designed to provide **high availability and partition tolerance** but there can be periods where the data on secondary nodes is not yet fully consistent with the primary node, due to the **asynchronous nature of replication.** This is often referred to as "**eventual consistency**". An AP in the CAP theorem.

To manage this trade-off, MongoDB provides a feature called "**read concern**". This allows you to specify how MongoDB should balance **consistency versus availability** when performing read operations. For example, a read concern of "**local**" allows for the most available (**faster**) reads but may return data that is stale, whereas a read concern of "**majority**" **will only return data that has been confirmed as written to a majority of replica set members**, offering a **higher level of consistency.**

MongoDB also provides "**write concern**" options that allow you to **control** the **level** of **acknowledgement** required from replica set members for **write operations**. For example, a **write concern of "1"** indicates that the write operation should return as soon as it has been acknowledged by the primary, **without waiting for any secondaries to replicate it.** A **write concern of "majority"** indicates that the write operation should not return until it has been **acknowledged by a majority of replica set members.**

Sharding

Universidad Francisco de Vitoria

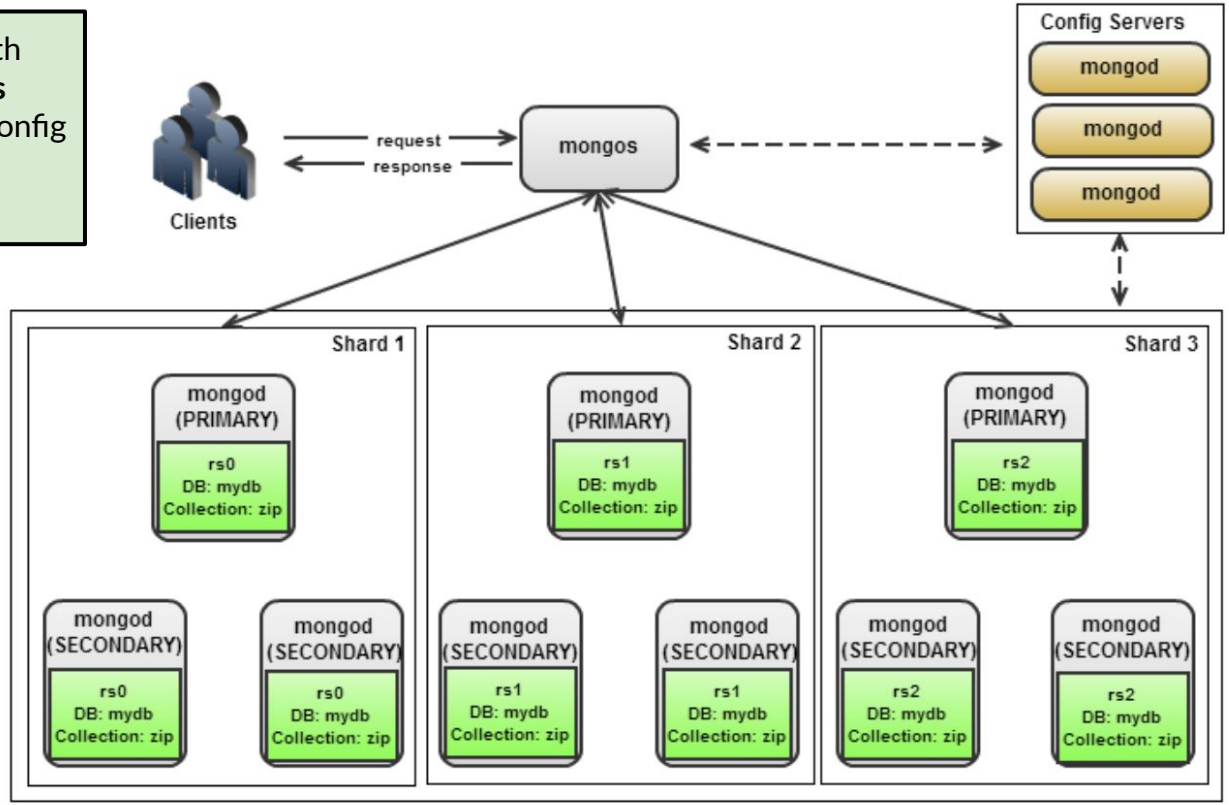
UFV Madrid

Sharding is meant to scale horizontally, a single node can be small to hold a big collection, so keys would be distributed among machines holding each a partitioning of the data

when Sharding is in place clients speak with mongos that is a **load balancer** that **knows where keys are** using a replica set called config server and **direct the client to the correct mongod** in order to do the query

Sharding is configured at collection level, so in a database some big collections could be sharded and other small collections don't

In a production environment this would be the norm and requires a lot of **configuration** and **maintenance**. This is why a Cloud option like Atlas for a SME in general will be probably the most economic option.



Atlas

Universidad Francisco de Vitoria

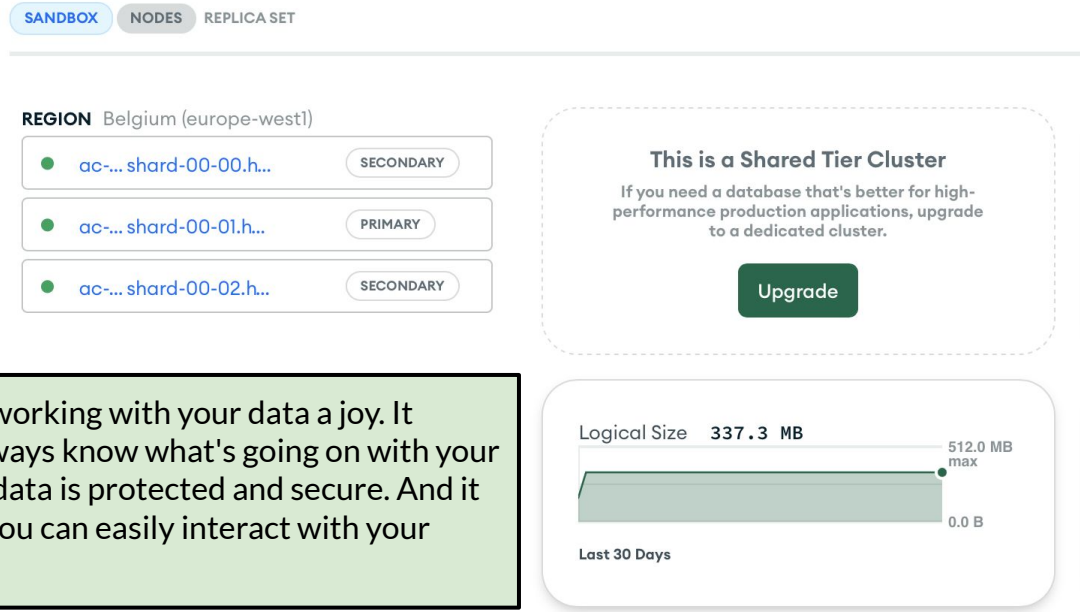
UFV Madrid

MongoDB Atlas is a **fully-managed cloud database service** that is elegant, flexible, and powerful, much like a well-composed symphony. It takes care of all the operational aspects of running MongoDB, providing users the freedom to focus on their data and application. The service manages the deployment of your databases on the cloud provider and region of your choice, and handles all the tedious aspects of database operations like backup, recovery, failure detection, and repair, much like a trustworthy and diligent conductor manages an orchestra.

With its **free tier**, known as the M0 tier, beginners and those wanting to experiment can start playing with MongoDB at no cost. This free tier offers 512 MB of storage and is an excellent way to get started with MongoDB without making any financial commitment, akin to dipping your toes in the water before diving in.

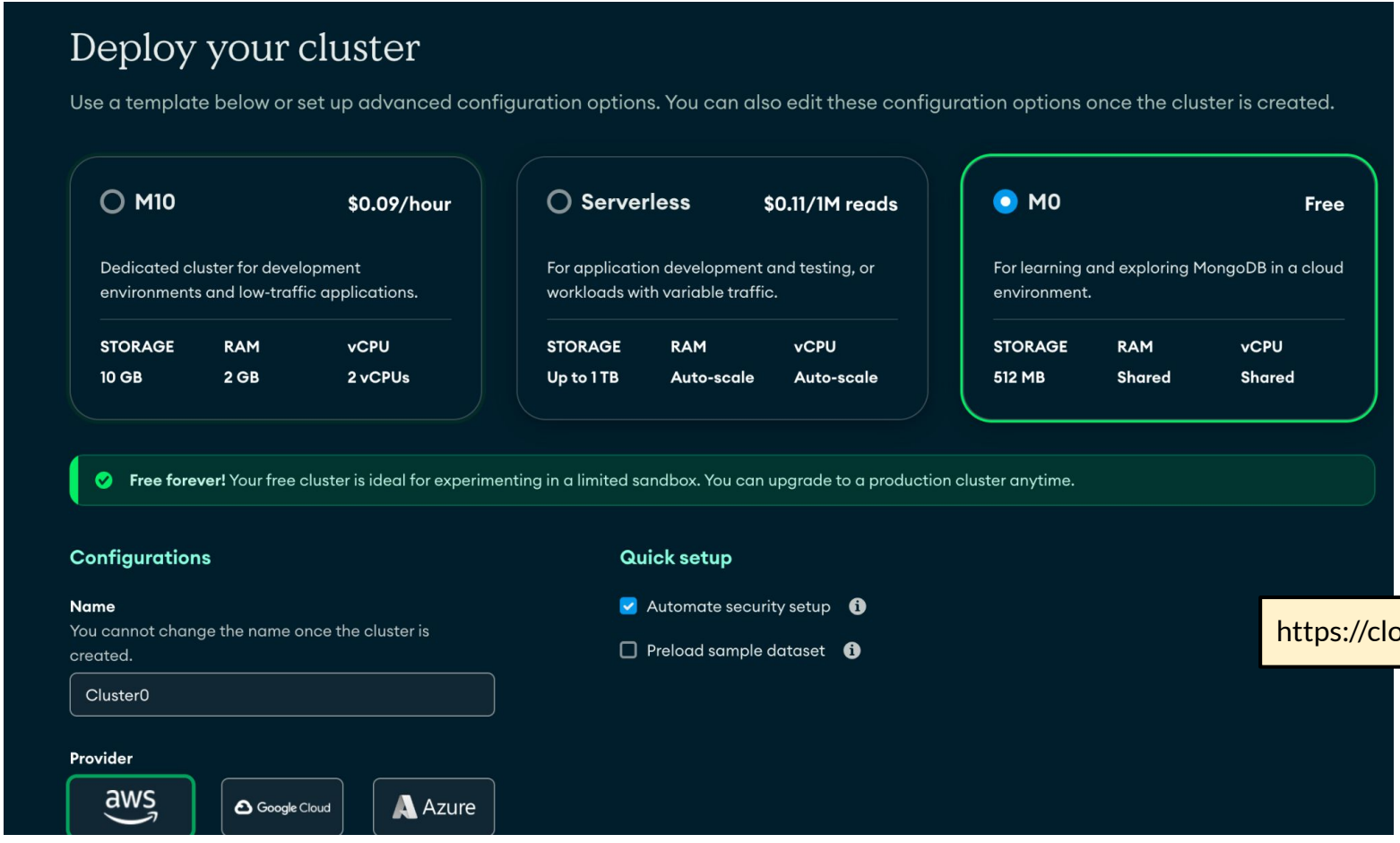
As your needs grow, Atlas offers a range of tiers, from the free M0 tier up to the M700 tier, with varying levels of storage, RAM, and other features.

Whether you're a small startup or a large enterprise, MongoDB Atlas has the flexibility to match your needs



Atlas provides a variety of tools and services that make working with your data a joy. It provides comprehensive monitoring and alerts, so you always know what's going on with your data. It provides built-in security features, ensuring your data is protected and secure. And it provides tools for data exploration and management, so you can easily interact with your data.

In Atlas use M0 free tier for learning purposes

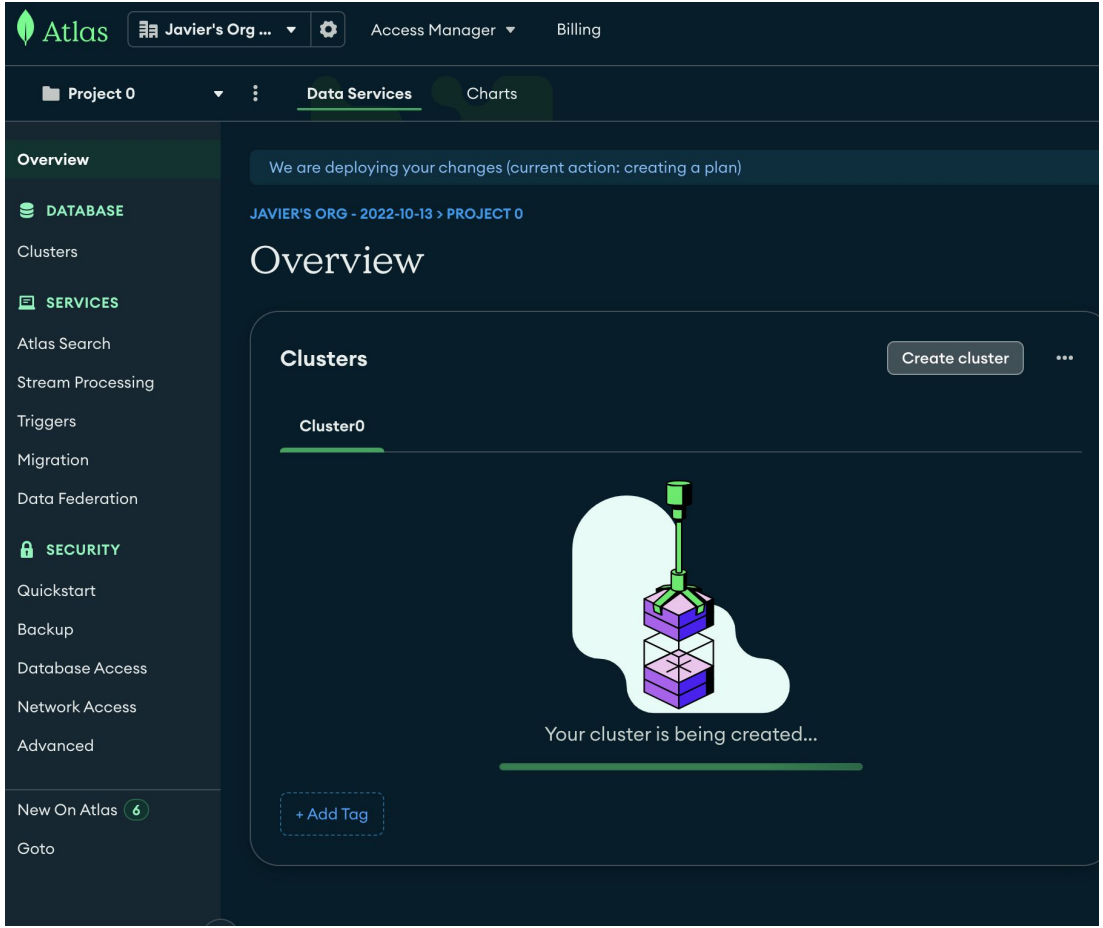


https://cloud.mongodb.com/

Universidad Francisco de Vitoria

UFV Madrid

Cluster being created



URI String

Universidad Francisco de Vitoria
UFV Madrid

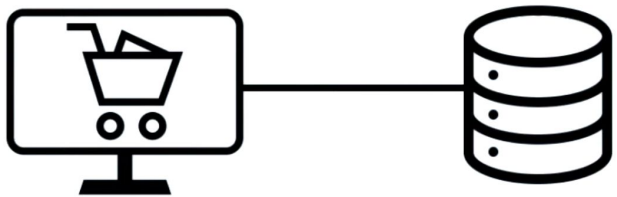
A URI is a string of characters that unambiguously identifies a particular resource. To use an analogy, it's like the address of a house. It tells you exactly where to go to find that house, just as a URI tells your software **exactly where** to go to find a specific **resource**.

When you see something like http://www.mywebsite/myresource.pdf, it's a URI that points to a specific resource (in this case, a PDF) located on a specific website. The **http** part is the **scheme**, which tells your browser what **protocol** to **use** to **access** the resource. In this case, it's telling your browser to use the **HTTP protocol.**

MongoDB Atlas is a **database as a service** offered by MongoDB, and it uses DNS SRV records to simplify the process of connecting to your MongoDB databases.

```
mongodb+srv://user:password@clusterURI.mongodb.net
/database
```

```
mongodb://username:password@
cluster0-shard-00-00-m220-lessons-mcxlm.mongodb.net:27017,
cluster0-shard-00-01-m220-lessons-mcxlm.mongodb.net:27017,
cluster0-shard-00-02-m220-lessons-mcxlm.mongodb.net:27017/mflix
```



DNS SRV records kind of a GW for us

Universidad Francisco de Vitoria
UFV Madrid

DNS SRV records are a **type of resource record in the Domain Name System (DNS)** that specifies **information** on **available services**. They are used to establish connections to services without requiring users to know the exact hostname and port number. Here's an example of how they're used.

```
_ldap._tcp.example.com. 86400 IN SRV 10 60 389 ldap1.example.com.
_ldap._tcp.example.com. 86400 IN SRV 20 60 389 ldap2.example.com.
_ldap._tcp.example.com. 86400 IN SRV 30 60 389 ldap3.example.com.
```

Imagine you have a service, for instance, an LDAP server, running on a cluster of machines in your network and you want clients to be able to connect to any one of these machines. You would set up SRV records in your DNS that might look something like this.

When a client wants to connect to your LDAP service, it queries the DNS for SRV records related to _ldap._tcp.example.com. The DNS server responds with the **list of records**, and the client uses the priority and weight information to decide which server to try to connect to first. If the first server doesn't respond, the client can try the next one, and so on.

In the case of MongoDB Atlas and the **mongodb+srv protocol**, the process is similar. When you create a cluster on MongoDB Atlas, **Atlas automatically sets up DNS SRV records for your cluster**. Your connection string includes a hostname like cluster0.mongodb.net, and when your **MongoDB driver** sees the mongodb+srv protocol, it knows to query the DNS for SRV records related to this hostname. The DNS server responds with the details of the servers in your Atlas cluster, and your **driver uses this information to establish a connection.**

HTTP vs SRV

Universidad Francisco de Vitoria
UFV Madrid

With HTTP, you typically need to know the exact URL of the resource you want to access like http://www.example.com/my-webpage. The URL includes the protocol (**http**), the domain name (**www.example.com**), and the path to the resource (**/my-webpage**).

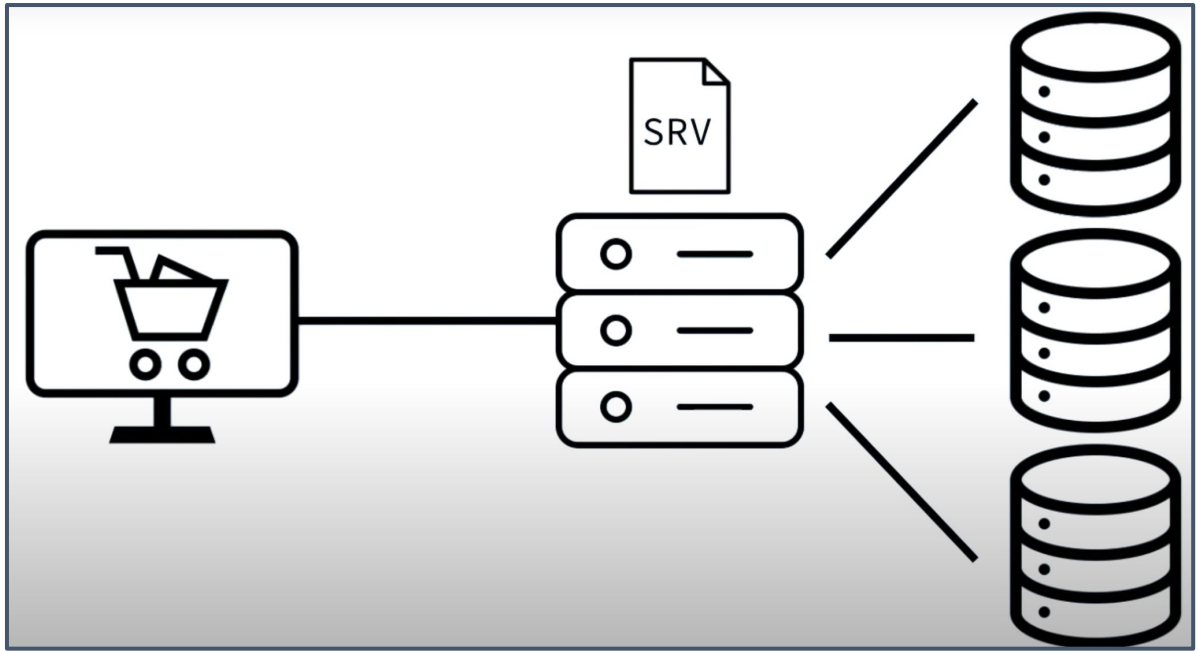
Now, imagine you're running a service, like a database or an email server, and this **service could be** hosted on any one of **several** different **machines**. If you were using the HTTP approach, you'd need to know the exact hostname and port number for the machine you want to connect to, and if that machine went down or you wanted to **distribute** your **connections** across **multiple machines** for **load balancing**, you'd need to **keep track** of all these details yourself. This can quickly become **complex and difficult to manage.**

This is where DNS SRV records come in. Instead of needing to know the exact hostname and port number, you just need to know the name of the service you want to connect to and the domain it's in. The DNS system takes care of the rest.

When you make a request to a service using a DNS SRV record, like _myservice._tcp.example.com, your client software (like your MongoDB driver or your email client) asks the DNS system for the SRV record associated with this service. The DNS system responds with a list of hostnames and port numbers for machines that provide this service. This list can include multiple entries, each with its own priority and weight, so your client can try to connect to the most desirable server first and fall back on others if the first one doesn't respond. all handled automatically by your client SW, so you don't have to keep track of all these details yourself. You just need to know the name of the service and the domain it's in, and you can rely on the DNS system to direct you to a server.

URI srv String

```
mongodb+srv://user:password@clusterURI.mongodb.net
/database
```



Like a DNS server, **keeps track on where databases are**, if some **server changes** we are still connected as srv record keeps track and **srv record** is **updated** to **reflect changes**

DNS SRV records simplify the process of connecting to distributed services. They allow you to connect to a service without needing to know the exact details of where it's hosted, and they provide built-in mechanisms for load balancing and failover. This is why they're used by MongoDB Atlas and many other distributed systems. They make your life easier.

Mongo DB Create

Universidad Francisco de Vitoria
UFV Madrid

```
show dbs
```

This will list all databases that have at least one collection with at least one document. Newly created databases without any data won't appear in this list.

```
use myNewDatabase
```

If myNewDatabase exists, this command will switch to it. If it doesn't, MongoDB will **create** it in **memory** for the duration of your session. However, myNewDatabase still **won't appear in the list generated by show dbs until you add some data.**

In MongoDB, a **collection** is akin to a table in SQL databases. A **document** in MongoDB is akin to a row in SQL. You can create a collection and insert a document at the same time using the insert() command:

```
db.myNewCollection.insertOne({name: 'Test', value: 'Hello, World!'})
```

After running, you can now list the databases again using **show dbs** and see that myNewDatabase is now in the list.

Note that MongoDB is schema-less, providing flexibility in the types of data stored in collections but this also means your application should handle data consistency.

When you initially install and start MongoDB, it does not come with any datasets. The databases you see – **admin** (related to system-wide users and **roles**), **config** (in sharded clusters to store metadata about the **cluster**), and **local** (In a replica set, this is where the oplog (operations log) resides. The local database is not replicated.) – are **system databases** that MongoDB uses for internal purposes. If you want to populate MongoDB with a testing dataset, you'll have to do so manually…

```
mongoimport --db myNewDatabase --collection myNewCollection --file dataset.json
```

you have a dataset file (for instance, in JSON format), you can import it using the mongoimport utility

insertOne and insertMany

Universidad Francisco de Vitoria
UFV Madrid

insertOne() and insertMany() are methods in MongoDB used for inserting data into a database. The **insertOne**() method is used when you want to **insert a single document**, and the **insertMany**() method is used when you want to **insert multiple documents at the same time**.

```
db.books.insertMany([
 {
  title: 'Moby Dick',
  author: 'Herman Melville',
  year: 1851
 },
 {
  title: 'Pride and Prejudice',
  author: 'Jane Austen',
  year: 1813
 },
 {
  title: '1984',
  author: 'George Orwell',
  year: 1949
 }
])
```

You've received a shipment of several new books. Instead of adding them one by one, you want to add them all at once.

```
db.books.insertOne({
 title: 'The Great Gatsby',
 author: 'F. Scott Fitzgerald',
 year: 1925
})
```

Like a librarian who wants to add a new book to your database.

Find, query, projection, limit

Universidad Francisco de Vitoria
UFV Madrid

In MongoDB, the find method is the primary way to query data. Just as you would use a searchlight to uncover items in a dark room, the **find command** illuminates the documents in a MongoDB collection that match certain criteria.

In MongoDB, a call to find typically takes the form **db.collection.find(query, projection)**, where db is your database, collection is the collection you're querying, query is the filter criteria for the documents you're looking for, and projection is the specification of which fields to return.

db.collection, the "namespace" is a full-qualified name that includes both the name of the database (db) and the name of the collection and uniquely identifies the collection within MongoDB.

query is a filter that specifies the criteria for the documents you want to find. For example, {author: "J.K. Rowling"} would find all documents where the author field is "J.K. Rowling". If you provide an empty document {}, MongoDB will return all documents in the collection.

projection: This is an optional parameter that specifies **which fields you want to be returned in the resulting documents.** For instance, **{title: 1, _id: 0}** will return documents **with only the title field (and exclude the _id field)**. If you omit the projection parameter, MongoDB will return all fields in the matched documents.

```
db.collection.find(query, projection).sort({title: 1}).limit(5)
```

there's an options parameter that can be used to control things like sorting, limiting, and skipping within the result set.

This would sort the resulting documents by the title field in **ascending** order and **limit** the output to the first 5 documents.

The flexibility of the find command is part of what makes Mongo such a powerful tool for working with data. Whether you need to find a single document, all documents that match certain criteria, or just a handful of documents while excluding certain fields…

Basic Find Queries (Select)

Universidad Francisco de Vitoria
UFV Madrid

```
db.collection.find({})
```
retrieves **all** documents in the collection.

```
db.collection.find({name: "John"})
```
This command returns all documents where the name field exactly matches "John".

```
db.collection.find({name: "John", age: 30})
```
returns all documents where the name is "John" AND the age is 30. Multiple fields in a query create an implicit AND condition.

```
db.collection.find({$or: [{name: "John"}, {age: 30}]})
```
All documents where the name is "John" OR the age is 30. The $or operator is used to specify a logical OR condition for two or more expressions.

```
db.collection.find({age: {$gt: 30}})
```
documents where the age is greater than 30. MongoDB provides a range of comparison operators, including $**gt** (greater than), $**gte** (greater than or equal to), $**lt** (less than), $**lte** (less than or equal to), and $**ne** (not equal to).

```
db.collection.find({name: {$regex: "^Jo"}})
```
This command returns all documents where the name starts with "Jo". The $regex operator allows you to search for a pattern in a string field.

Cursor

Universidad Francisco de Vitoria
UFV Madrid

The **find command returns a cursor**. You can imagine a cursor like a magical book marker that remembers where you are in a list. Even if you were reading through a library of thousands of books, a cursor would allow you to keep your place, moving forward and backward through the collection at your leisure.

```
let cursor = db.books.find({author: "J.K. Rowling"});
while (cursor.hasNext()) {
  print(tojson(cursor.next()));
}
```

cursor.hasNext() checks if there are more books to read, and cursor.next() takes you to the next book.

Main cursor methods:

1. **limit(number):** Restricts the result set to the specified number of documents.
2. **sort({field: order})**: Sorts the result set based on the specified field(s). 'order' is 1 for ascending and -1 for descending.
3. **skip(number):** Skips the specified number of documents in the result set.
4. **project({field: 1 or 0})**: Selectively includes or excludes fields from documents in the result set.
5. **count()**: Returns the count of documents that match the query.
6. **toArray()**: Converts the result set into an array of documents.
7. **forEach(function):** Applies a JavaScript function to each document in the result set.
8. **next():** Returns the next document in the result set.
9. **hasNext()**: Checks if there are more documents in the result set.
10. **explain()**: Provides statistics about query performance.

Add documents, _id key field

Universidad Francisco de Vitoria
UFV Madrid

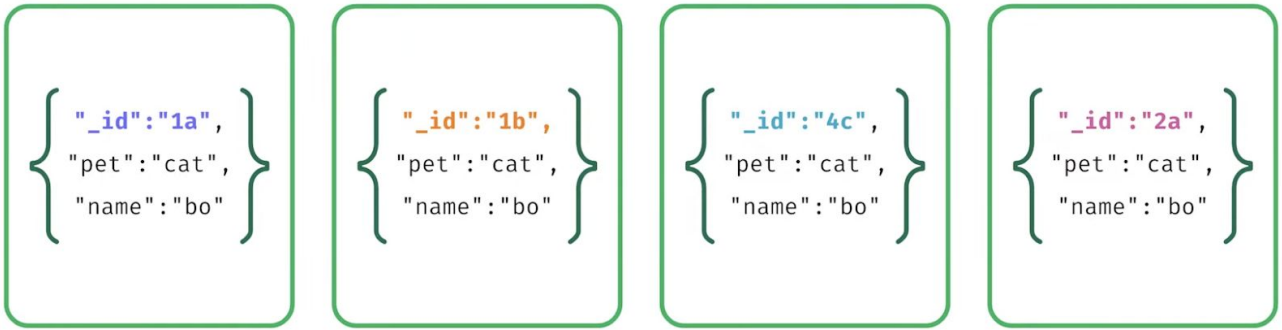
In MongoDB, every document must have a unique _id field that acts as a primary key. If an inserted document omits the _id field, MongoDB automatically generates an **ObjectId** for this field.

ObjectId: An ObjectId is a 12-byte identifier typically constructed using:

- A **4-byte timestamp,** representing the creation's seconds portion.
- A **5-byte random value.**
- A **3-byte incrementing counter**, initialized to a random value.

This structure ensures the probability of collisions is extremely low. Hence, it's usually safe to let MongoDB handle _id generation, and is the standard practice.

Custom _id: Although not common or recommended, you can supply your own **_id** when inserting documents. This can be of any type, such as String, Number, etc., as long as it's unique within the collection.



It's crucial to ensure the uniqueness of _id, as violating this will lead to a **DuplicateKey** error.

One might want to use custom _ids for specific use-cases, like linking documents across collections or databases, or having a business-specific identifier. Yet, keep in mind the **responsibility for uniqueness then lies on you**, and incorrectly managed, can lead to **data integrity issues.**

Add documents

Universidad Francisco de Vitoria
UFV Madrid

Mongo and in general NoSQL databases gives us a ton of **flexibility** in contrast to RDBMS rigid structures

At the same time it helps us auto generating a _id

```
db.inspections.insert({
    "id" : "10021-2015-ENFO",
    "certificate_number" : 9278806,
    "business_name" : "ATLIXCO DELI GROCERY INC.",
    "date" : "Feb 20 2015",
    "result" : "No Violation Issued",
    "sector" : "Cigarette Retail Dealer - 127",
    "address" : {
        "city" : "RIDGEWOOD",
        "zip" : 11385,
        "street" : "MENAHAN ST",
        "number" : 1712
    }
})
```

The document **structure is totally free** while APIs could help us force a schema and in corporate environments this is quite normal.

But in general, a Key will have a BSON value associated with whatever fields we decided to include in the document, not very stable at high scale but Mongo gives us the flexibility

Update operations

Universidad Francisco de Vitoria
UFV Madrid

```
db.collection.updateOne(filter, update, options)
```

filter: to know where to update, **update**: the kind of update that needs to happen.

```
db.zips.updateMany({ "city": "HUDSON" }, { "$inc": { "pop": 10 } })
```
→ increment 10 pop field of every city named HUDSON

$inc" would be an update operator in MQL, Mongo Query Language

```
db.grades.updateOne(
  { "student_id": 250, "class_id": 339 },
  { "$push": { "scores": { "type": "extra credit",  "score": 100 } } }
)
```

updateOne is **atomic** on the level of a **single document** (all changes to the document are either fully applied or not applied at all) → either the new score will be pushed to the scores array successfully, or the document will remain unchanged if an error occurs.

Here with $push we **add an element to scores array, if the array still is not present in the document, it is created with this operation.**

The new element being added to this scores array is an object: { "type": "extra credit", "score": 100 } that represents a score item with two properties: type and score, type is set to "extra credit", score is set to 100.

Filter Criteria: { "student_id": 250, "class_id": 339 } → The method will look for a document in the grades collection where student_id is 250 and class_id is 339. The operation will be performed on the **first document that matches** these criteria.

other update commands

Universidad Francisco de Vitoria
UFV Madrid

```
db.collection.updateOne(
 { name: "John" },
 { $set: { age: 31 } }
)
```

This command **updates** the **age field** of the **first document where** the **name** is "**John**". **If age doesn't exist, it's added to the document.**

```
db.collection.updateMany(
 { name: "John" },
 { $set: { age: 31 } }
)
```

This command updates the age field for **all documents** where the name is "John"

```
db.collection.replaceOne(
 { name: "John" },
 { name: "John", age: 31, city: "New York" }
)
```

note that this new document completely replaces the old one. If the old document had additional fields that are not included in the replacement document, those fields will be lost.

If no document matches the filter criteria, replaceOne will not create a new document. If you need a replacement operation that also creates a document if none matches the filter, you would use the **upsert** option.

update Arrays

Universidad Francisco de Vitoria
UFV Madrid

When updating array fields, MongoDB provides several operators:

- $**push**: Adds an item to an array.
- $**pop**: Removes an item from an array (either the first or last item).
- $**pull**: Removes all instances of a value from an array.
- $**addToSet**: Adds a value to an array unless it already exists.

```
db.collection.updateOne(
 { name: "John" },
 { $push: { hobbies: "Reading" } })
```

This command adds "Reading" to the hobbies array for the first document where name is "John".

```
db.collection.updateOne(
 { name: "John" },
 { $pop: { hobbies: -1 } })
```

This command removes the first item from the hobbies array where name is "John". Use 1 to remove the last item.

```
db.collection.updateOne(
 { name: "John" },
 { $pull: { hobbies: "Reading" } })
```

This command removes "Reading" from the hobbies array where name is "John".

```
db.collection.updateOne(
 { name: "John" },
 { $addToSet: { hobbies: "Reading" } })
```

This command adds "Reading" to the hobbies array where name is "John", but only if "Reading" is **not already in** the array.

upsert

Universidad Francisco de Vitoria
UFV Madrid

If you want MongoDB to create a new document when no document matches the update query criteria, you can **set the upsert option to true**. In this case, MongoDB **creates** a **new document** using both the query criteria and the specified update.

```
db.collection.updateOne(
 { name: "John" },
 { $set: { age: 31 } },
 { upsert: true }
)
```

If there is no matching documents, updateOne, updateMany, and replaceOne operations will not modify any documents. However, no error is thrown and the operation is considered successful, but the count of modified documents (as reported in the response) is zero.

```
db.collection.updateOne(
 { name: "John" },
 { $push: { hobbies: "Reading" } }
)
```

If there is no hobbies array field in the document where name is "John":
If you use $**push**, $**addToSet**, or $**pop**, and the hobbies field doesn't exist in the targeted document, MongoDB will **add** an **array field** with the specified value (for $push and $addToSet).

Upsert is useful when you want MongoDB to create a new document as part of an update operation if the document doesn't already exist.

increment, decrement, multiply, divide

Universidad Francisco de Vitoria
UFV Madrid

The $inc operator increments a field by a specified value. This operator can work with both positive and negative values. If the field does not exist, $inc will create it and set it to the specified value.

```
db.collection.updateOne(
  { name: "John" },
  { $inc: { age: 1 } }
)
```

This operation increments the age of "John" by 1. If "John" doesn't exist or if age isn't present in the "John" document, MongoDB will create the document or the field respectively, and set the age to 1.

```
db.collection.updateOne(
  { name: "John" },
  { $inc: { age: -1 } }
)
```

There isn't a specific $dec operator in MongoDB, but you can use $inc with a negative value to achieve the same effect.

```
db.collection.updateOne(
  { name: "John" },
  { $mul: { age: 2 } }
)
```

This operation doubles John's age. If the age field is absent in the document, $mul will create it and set it to 0.

Other useful operations

```
db.collection.updateOne(
 { name: "John" },
 { $rename: { "age": "years" } }
)
```

This operation renames the age field to years in John's document. If the age field doesn't exist, no operation is performed.

```
db.collection.updateOne(
 { name: "John" },
 { $unset: { age: "" } }
)
```

This operation removes the age field from John's document.

```
db.collection.updateOne(
 { name: "John" },
 { $min: { age: 30 } }
)
```

This operation sets John's age to 30 if it's currently more than 30. Similar min operation

```
db.collection.updateOne(
 { name: "John" },
 { $currentDate: { lastModified: true } }
)
```

This operation sets John's lastModified field to the current date.

Task with update commands

Universidad Francisco de Vitoria
UFV Madrid

```
{
 "_id": ObjectId("5ec414e5e722bb1f65a25451"),
 "pet": "wolf",
 "domestic?": false,
 "diet": "carnivorous",
 "climate": ["polar", "equatorial", "continental", "mountain"]}
```

```
db.pets.updateMany({ "pet": "cat" },
        { "$push": { "climate": "continental",
                "look": "adorable" } })
```

An array “look” is created in each cat doc with “adorable” as first element

```
db.pets.updateMany({ "pet": "cat" },
        {"$set": { "domestic?": true, "diet": "mice" }})
```

Fields are just updated

```
db.pets.updateMany({ "pet": "cat" },
        { "$set": { "type": "dangerous",
                "look": "adorable" }})
```

"type" and "look" are created as are not in the original doc where pets are cats

```
db.pets.updateMany({ "pet": "cat" },
        { "$set": { "climate": "continental" }})
```

fields are just updated but note that the array is no longer in place for cats, just a single string

Deletion

Universidad Francisco de Vitoria
UFV Madrid

```
db.inspections.deleteMany({ "test": 1 })
db.inspections.deleteOne({ "test": 3 })
```

```
db.inspection.drop()
```

When all collections are dropped from a database, the database no longer appears in the list of databases when you run show dbs.

So, no longer exists

Install on Mac

Universidad Francisco de Vitoria

UFV Madrid

brew services list is a command you can run on a macOS system if you've installed the **Homebrew package manager.**

It **lists all the services that you've installed through Homebrew** and their current status (started, stopped, etc.). These services are usually background processes or daemons that provide some kind of functionality, like a web server or a database server.

install Homebrew by pasting the following command in your terminal:
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

```
brew tap mongodb/brew
brew install mongodb-community
```

Once Homebrew is installed, you can install MongoDB with the command

Mac Homebrew Package Manager

Universidad Francisco de Vitoria
UFV Madrid

Homebrew is a package manager specifically designed for macOS, although it can also be used on Linux. It simplifies the process of installing, updating, and managing software on your computer.

To illustrate the concept, let's consider an analogy. Think of your computer as a house and software applications as different appliances or furniture you might want to add to the house. Without a package manager, if you want to add a new piece of furniture (software), you would need to manually **go to the store** (software's **website**), pick out the furniture (**download** the software), bring it home, and **put it together following all the instructions** (**install** and **configure** the **software**).

A **package manager** like **Homebrew** acts as a **personal shopper and handyman.** You just tell it what you want (e.g., brew install mongodb), and it goes to the store for you, picks out the right item, brings it home, and sets it up correctly. It ensures that the **right versions** are installed, and **dependencies** (other pieces of software that your software relies on) are also taken care of.

In the software world, there are many package managers. Some are designed for specific **languages** like **npm** for Node.js, **pip** for Python, and **gem** for Ruby. Others are built for **operating systems**, like **apt** for Ubuntu, **yum** for CentOS, and **Homebrew** for macOS. Homebrew stands out in the macOS world because macOS doesn't come with its own native package manager.

Let's say you wanted to install the **wget** utility (a tool for downloading files from the web). Without Homebrew, you would need to find a **reliable source** online, download the correct version for your system, and **go through the installation process manually**. With Homebrew, you simply open a terminal and type **brew install wget**, and Homebrew takes care of the rest.

Homebrew also makes it easy to manage your installed software. You can upgrade all your software to the latest versions with **brew upgrade**, or remove software with **brew uninstall**. The **brew services list** command shows you all the background services you've installed with Homebrew and their current status.

brew tap

Universidad Francisco de Vitoria
UFV Madrid

The brew tap command in Homebrew is used to **add** additional **repositories** of software.
By default, when you use Homebrew to install software, it looks in the **main Homebrew repository** on GitHub. However, many developers or organizations maintain **their own repositories** with their own software, or versions of software not available in the main Homebrew repository. These are called "**taps**".

The term "tap" is a fun metaphor borrowed from the world of beer brewing. In a brewery or bar, a "tap" is a device that controls the release of beer from a keg. When you want to serve a certain type of beer, you "tap" the corresponding keg. If you have more types of beer, you have more taps.

a "tap" represents a source of software packages, like a keg represents a source of beer. When you "tap" a repository, you are essentially making the software in that repository available for installation, just like tapping a keg makes the beer inside available to be served.

```
brew tap username/repo
```

To see a list of all currently tapped repositories, you can use the **brew tap** command with no arguments. To untap a repository, you can use the **brew untap** command followed by the repository identifier.

So, think of your macOS system as a bar for software. The main Homebrew repository is like the bar's main selection of beer. But using the brew tap command allows you to add more "kegs" (repositories) of "beer" (software) to your "bar" (system). This gives you access to a wider variety of software, much like a bar with more taps can serve a wider variety of beer. It's a playful way to describe extending the functionality of the Homebrew package manager.

Windows Chocolatley

Universidad Francisco de Vitoria

UFV Madrid

Chocolatey allows you to manage your software packages with a similar level of finesse and simplicity.

To bring Chocolatey into your life, you need to install it first. Don't worry, it's as simple as melting chocolate. Open your PowerShell as an administrator and paste this command into it:
Set-ExecutionPolicy Bypass -Scope Process -Force; [System.Net.ServicePointManager]::SecurityProtocol = [System.Net.ServicePointManager]::SecurityProtocol -bor 3072; iex ((New-Object System.Net.WebClient).DownloadString('https://chocolatey.org/install.ps1'))

choco install git	Chocolatey will fetch the right version, set it up, and even clean up after itself. It's like having a personal software butler at your service.
choco upgrade git	Updating software is easy. Let's say you want to update Git to the latest version. You don't need to check the Git website or do any manual downloading. Just ask Chocolatey to do it.
choco list --local-only	This will list all the packages you've installed with Chocolatey.

It's an effortless and efficient way to manage software on Windows, making your life just a bit sweeter (chocolate).

Linux Package Managers

Universidad Francisco de Vitoria

UFV Madrid

On a Linux system, the package manager is your guide to this marketplace, helping you search, install, update, and remove software with ease and precision.

There are several popular package managers based on the Linux distribution you're using.

For Debian-based systems like Ubuntu, you'll use **apt**. For Red Hat-based systems like Fedora, you'll use dnf or **yum**. And for Arch Linux, you'll use **pacman**.

Let's take a look at apt, the package manager for Ubuntu, one of the most popular Linux distributions. Imagine you've just set up your Ubuntu system and you want to install the text editor vim. No need to search the web or worry about downloading the right version. Just open your terminal and enter: **sudo apt install vim**

```
sudo apt update
sudo apt upgrade vim
```

The first command updates the list of available packages and their versions, while the second upgrades vim to its latest version.

```
apt list --installed
```

If you're ever curious about what packages you have installed,

Navigating the world of Linux package management is an adventure filled with discovery and ease. It's a testament to the power and flexibility of Linux.

mongodb native driver

Universidad Francisco de Vitoria
UFV Madrid

The **MongoDB native driver** is an extremely versatile tool that allows Node.js applications to interact with MongoDB databases. It offers a rich API for querying, indexing, and aggregation.

```
const { MongoClient } = require('mongodb');
const uri = 'mongodb://localhost:27017';
const client = new MongoClient(uri, { useUnifiedTopology: true });

async function main() {
  try {
    await client.connect();

    console.log('Successfully connected to the database');
  } catch (err) {
    console.error('Failed to connect to the database', err);
  } finally {
    await client.close();
  }
}

main().catch(console.error);
```

You can use the MongoClient class to create a connection to your MongoDB database. Note that the connection should ideally be created once and then reused throughout your application.v

async await

Universidad Francisco de Vitoria
UFV Madrid

The **async/await** pattern in JavaScript is a way to write asynchronous code in a more readable, synchronous-like manner. **Asynchronous code** is a **fundamental** part of **JavaScript** and is heavily **used when dealing with operations that take an unknown amount of time to complete,** like **requests** to a **database** or a **server**.

Imagine you're cooking a fancy dinner. Some tasks need to complete before others can begin - you can't start cooking until you've shopped for ingredients, and you can't serve the food until it's cooked. **Cooking is asynchronous** - you start the oven, but you don't stand there and wait for it to preheat. You do other things while you wait.

```
async function cookDinner() {
  try {
    console.log('Start shopping');
    await shopForIngredients(); // Asynchronous function
    console.log('Start cooking');
    await cookMeal(); // Asynchronous function
    console.log('Dinner is ready. Time to eat!');
  } catch (error) {
    console.log('Something went wrong in dinner', error);
  }
}
cookDinner();
```

"Shopping" and "cooking" are **asynchronous functions** - they take time to complete, but you can do other things while you wait for them to finish.

"**Start cooking**" is like the **await** keyword - you can't start this task until the previous one (shopping) is complete.

"**Serving the food**" is like the rest of your code that **runs after your await statements.** It r**uns after all the awaited tasks are complete**.

cookDinner is an async function. Inside this function, we await for shopForIngredients and cookMeal. These could be any asynchronous operations - making a request to a server, querying a database, reading a file, etc.

async await 2


```
const { MongoClient } = require('mongodb');
const uri = 'mongodb://localhost:27017';
const client = new MongoClient(uri, { useUnifiedTopology: true });

async function fetchUsers() {
  try {
    await client.connect(); // Connect to MongoDB

    const db = client.db('mydatabase'); // Select the database
    const users = db.collection('users'); // Select the collection

    const allUsers = await users.find().toArray(); // Fetch all users

    console.log(allUsers); // Log the users
  } catch (error) {
    console.error('Error fetching users:', error);
  } finally {
    await client.close(); // Close the connection
  }
}

fetchUsers();
```

fetchUsers is an async function that connects to a MongoDB database, fetches all users, and logs them.

We use await before **client.connect(), users.find().toArray(),** and **client.close()** because these are asynchronous operations - they return Promises.

fetchUsers() is called at the top level of the script. In other words, it's not being called inside another async function. As of now, **await can only be used inside async functions.** If you try to use await at the top level of a script (outside of any async function), you'll get a syntax error.

fetchUsers() is called without await. This means that the script won't wait for fetchUsers to finish before it exits. If fetchUsers was doing something more substantial (like starting a web server), the script would exit before fetchUsers had a chance to finish.

Inserting documents into a collection

```
const { MongoClient } = require('mongodb');
const uri = 'mongodb://localhost:27017';
const client = new MongoClient(uri, { useUnifiedTopology: true });

async function main() {
  try {
    await client.connect();
    const database = client.db('test');
    const collection = database.collection('documents');

    // Insert a single document
    const doc = { name: 'John', age: 30, occupation: 'Engineer' };
    const result = await collection.insertOne(doc);
    console.log(`Successfully inserted document with _id:
${result.insertedId}`);
  } catch (err) {
    console.error('Failed to insert document', err);
  } finally {
    await client.close();
  }
}
main().catch(console.error);
```

Once connected, you can use the db method on the MongoClient instance to get a reference to a database, and the collection method to get a reference to a collection.

Universidad Francisco de Vitoria
UFV Madrid

Querying documents from a collection

Universidad Francisco de Vitoria
UFV Madrid

```
const { MongoClient } = require('mongodb');
const uri = 'mongodb://localhost:27017';
const client = new MongoClient(uri, { useUnifiedTopology: true });

async function main() {
  try {
    await client.connect();
    const database = client.db('test');
    const collection = database.collection('documents');

    // Query for all documents where age is greater than 25
    const query = { age: { $gt: 25 } };
    const cursor = collection.find(query);

    // Print all matching documents
    await cursor.forEach(doc => console.log(doc));
  } catch (err) {
    console.error('Failed to query documents', err);
  } finally {
    await client.close();
  }
}
```

```
import asyncio
from motor.motor_asyncio import AsyncIOMotorClient

async def main():
    client = AsyncIOMotorClient('mongodb://localhost:27017')
    db = client['testdb']
    collection = db['test']

    # find
    docs = await collection.find({'name': 'John'}).to_list(length=100)
    for doc in docs:
        print(doc)

    # update
    await collection.update_one({'name': 'John'}, {'$set': {'age': 30}})

asyncio.run(main())
```

Mongoose

Universidad Francisco de Vitoria
UFV Madrid

```
const mongoose = require('mongoose');
mongoose.connect('mongodb://localhost:27017/myproject');

const Schema = mongoose.Schema;
const UserSchema = new Schema({
  name: String,
  age: Number
});

const User = mongoose.model('User', UserSchema);

const john = new User({ name: 'John', age: 30 });

john.save(function(err, john) {
  if (err) return console.error(err);
  console.log(john);
});
```

```
const MongoClient = require('mongodb').MongoClient;
const uri = "mongodb://localhost:27017/myproject";
MongoClient.connect(uri, function(err, client) {
  const collection =
client.db("myproject").collection("documents");
  // Insert a document
  collection.insertOne({
    name: 'John',
    age: 30
  }, function(err, result) {
    if(err) console.log(err);
    console.log(result);
    client.close();
  });
});
```

you can see that Mongoose introduces some extra steps, like creating a schema and a model; these steps provide additional power and flexibility, such as the ability to define **default** values, custom **validation** rules, virtuals, etc. The MongoDB native driver, on the other hand, is simpler but does not provide these features out-of-the-box.

Understand your project requirements and choose the tool that best meets your needs. The MongoDB native driver is great for simple tasks and when you need more control, while **Mongoose** is great for more **complex** projects and when you need more out-of-the-box features.

Mongodb driver creates crazy code, mongoose brings order



MongoDB.

mongoose

Pluralization in Mongoose

Universidad Francisco de Vitoria
UFV Madrid

```
const mongoose = require('mongoose');
const Schema = mongoose.Schema;

const userSchema = new Schema({
  name: String,
  age: Number
});
const User = mongoose.model('User', userSchema);
```

Here, Mongoose will automatically **map** the **'User' model** to the **'users' collection** in your MongoDB database.

```
let john = new User({ name: 'John', age: 30 });
john.save((err, john) => {
  if (err) return console.error(err);
  console.log("Document saved successfully!");});
```

John's details would be saved in the **'users' collection**, not 'User', thanks to Mongoose's pluralization.

Pluralization in Mongoose is an application of the linguistic concept of making singular nouns plural, implemented in a database context to create collection names.

Mongoose uses a library called **pluralize** to convert the singular model names to their plural form and uses the resulting name as the collection name. This is a convention employed by Mongoose to simplify the **model-to-collection mapping** process, and it's quite handy once you get accustomed to it.

Object Data Mapping, ODM

Universidad Francisco de Vitoria
UFV Madrid

An Object-Data Mapping (**ODM**) model, like **Mongoose**, is a design pattern in computer science that creates a **virtual object database** that can be used within the programming language.

Mongoose provides a straightforward, **schema-based solution** to model your application data with MongoDB. It includes built-in **typecasting**, **validation**, **query** building, and business logic hooks, which makes it a robust tool for data modeling. It also allows MongoDB documents to be used as if they were objects in your programming language, hence increasing the ease of interaction with the database.

```
let john = new User({ name: 'John', age: 30 });
john.age = 31;
john.save((err, john) => {
  if (err) return console.error(err);
  console.log("Document saved successfully!");
});
```

you're working with **john** as if it were a **regular JavaScript object**. You're able to modify its properties and save it back to the database

An Object Data Mapping (ODM) system like Mongoose can be very **liberating**. It **abstracts away** many of the details of working directly with a database, allowing you to **treat your data like regular JavaScript objects.**

With ODM we work object oriented style

Universidad Francisco de Vitoria
UFV Madrid

The instance method design choice allows for better encapsulation of data and behavior. In **object-oriented programming**, it's a common practice to **bundle data** (properties of the object) and the **behavior** (methods) **together**.

By calling .**save**() on an **instance** of a **model**, we're adhering to this principle. We're telling a **specific User to save itself,** and it knows how to do it because the **behavior** (the method .save()) is **bundled with** the **data** (the properties of the User).

```
db.collection('users').updateOne(
  { _id: 'someId' },
  { $set: { name: 'John', age: 31 } },
  function(err, result) {
    if (err) console.error(err);
    console.log(result); });
```

You're explicitly dealing with the 'users' collection, using MongoDB's updateOne method, and handling the raw result of the operation.

There's a clear awareness that you're working directly with a database.

ODM-2

Universidad Francisco de Vitoria

UFV Madrid

```
User.findById('someId', function (err, user) {
  if (err) console.error(err);
  user.name = 'John';
  user.age = 31;
  user.save(function (err, updatedUser) {
    if (err) console.error(err);
    console.log(updatedUser);});});});
```

Here, you're working with the user as if it were a simple JavaScript object.

You're fetching the user, updating properties directly, and saving the changes, all while Mongoose takes care of translating these operations into the appropriate database commands behind the scenes.

```
let newUser = new User({
  name: 'Alice',
  age: 25});

newUser.save(function (err, savedUser) {
  if (err) console.error(err);
  console.log(savedUser);});
```

The newUser object doesn't exist in the database until the save method is called.

The ODM approach is especially powerful because it allows you to perform complex database operations while maintaining the simplicity and familiarity of JavaScript object manipulation.

Callbacks, Promises and Async/Await

Universidad Francisco de Vitoria

UFV Madrid

Callbacks, Promises, and Async/Await are different ways of handling asynchronous operations in JS.

```
User.findById('someId', function (err, user) {
  if (err) return console.error(err);
  console.log(user);
});
```

Callbacks are the oldest method, and they involve passing a function (a "callback") into another function so it can be executed later. However, callbacks can lead to "callback hell" if there are multiple nested callbacks, making the code difficult to read and understand.

```
User.findById('someId')
  .then(user => console.log(user))
  .catch(err => console.error(err));
```

Promises were introduced as a solution to this problem. A Promise in JS is an object that may produce a single value some time in the future. It can be in one of three states: **fulfilled**, **rejected**, or **pending**. Promises allow for a cleaner, more readable code structure compared to callbacks.

```
try {
  const savedJohn = await john.save();
  console.log("Successfully saved", savedJohn);
} catch(err) {
  console.log("Something went wrong!", err);
}
```

Async/Await is syntactic sugar built on top of Promises, introduced in ES2017, and it allows you to write Promise-based code as if it were synchronous, but it executes asynchronously.

It makes asynchronous code look and behave a little more like synchronous code, making it easier to understand and reason about. It is now a widely accepted way of handling asynchronous operations in modern JavaScript.

Mongoose Read

Universidad Francisco de Vitoria
UFV Madrid

```
User.find({ age: { $gt: 20 } }, (err, users) => {
  if (err) {
    console.log('An error occurred:', err);
    return;}
  console.log('Users older than 20:', users);
});

User.find({ name: { $regex: /^Jo/ } }, (err, users) => {
  if (err) {
    console.log('An error occurred:', err);
    return;
  }
  console.log("Users whose name starts with Jo:", users);
});
```

In the first example, **{ age: { $gt: 20 } }** is a query object that matches all documents where the age field is greater than 20. $gt is a comparison query operator that stands for "greater than".

In the second example, **{ name: { $regex: /^Jo/ } }** is a query object that matches all documents where the name field starts with 'Jo'. $regex is a query operator that you can use to search for documents using regular expressions.

In Mongoose, the result of the find() method does not return a cursor like the native MongoDB driver. Instead, it returns an **array of documents** that match the specified filter.

Methods at schema level

Universidad Francisco de Vitoria
UFV Madrid

```
User.find({}, function(err, users) {
  if (err) {
    console.log(err);
  } else {
    users.forEach(user => {
      console.log(user.getFullName());
    }); }});
```

Because these **objects are instances of the Mongoose model,** they have access to any **instance methods** that you might define in your model schema.

```
UserSchema.methods.getFullName = function() {
  return this.firstName + ' ' + this.lastName;
};
```

Benefit → allows define methods at the schema level then available on all instances of the model, promoting code reusability and maintainability.

Validations Mongoose Schema level

Universidad Francisco de Vitoria

UFV Madrid

```
const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true,
    minlength: 5,
    maxlength: 100
  },
  age: {
    type: Number,
    required: true,
    min: 10,
    max: 20
  },
  email: {
    type: String,
    required: true,
    validate: {
      validator: function(v) {
        return /\S+@\S+\.\S+/.test(v);
      },
      message: props => `${props.value} invalid`
    }}});
const User = mongoose.model('User', userSchema);
```

name: It is required, and must be a **string** of at least **5 characters** and not more than **100 characters**.
age: It is required, and must be a **number** between **10 and 20**.
email: It is required and must be a **valid email**. The email validation uses a **custom function** that tests the email.

In addition to **built-in validators**, Mongoose supports **custom validation through the validate field** where you can provide your own validation function. This gives you the flexibility to implement complex validation logic as needed.

```
const user = new User({
  name: 'John',
  age: 22,  // Outside the allowed range (10 - 20)
  email: 'invalidemail',  // Not a valid email
});

user.save(function(err) {
  if (err) {
    console.log(err.message);  // Validation failed }});
```

Mongoose allows you to apply **validation** on your model's **schema** definitions. This is a **powerful feature** that **ensures** data **consistency**, and helps prevent errors that may occur due to incorrect or missing data.

updates with Mongoose

Universidad Francisco de Vitoria
UFV Madrid

```
User.updateOne({ name: 'John' }, { age: 25 },
function(err, res) {
 // Updated at most one doc });
```

updateOne() updates the first document that matches the provided filter.

```
User.updateMany({ name: 'John' }, { age: 25 },
function(err, res) {
 // Updated all docs matched});
```

updateMany() updates all documents that matches the provided filter.

`res.modifiedCount` contains the number of docs that MongoDB updated

```
User.findOneAndUpdate({ name: 'John' }, { age: 25 }, { new: true }, function(err, doc) {
 // `doc` is the document _after_ `update` was applied because of `{ new: true }`
});
```

findOneAndUpdate() finds a document via the provided filter, updates it according to the provided update object, and **returns the updated document** (in the callback)

```
User.updateOne({ name: 'John' }, { $inc: { age: 1 } },
function(err, res) {

});
```

increment the age of a user named 'John' by 1,

Mongoose supports a range of update operators, for example:

- $**set**: Set the value of a field in a document.
- $**inc**: Increment the value of a field by a specified amount.
- $**push**: Add an item to an array.

```
User.updateOne({ name: 'John' }, { $push: { emails:
'new_email@example.com' } }, function(err, res) { callback body });
```

add a new email **to** a user's email **list**

```
const User = mongoose.model('User', userSchema);
```

```
const userSchema = new
mongoose.Schema({
  name: String,
  age: Number,
  email: [String],
});
```

Delete with Mongoose

Universidad Francisco de Vitoria
UFV Madrid

In Mongoose, the **deleteOne**() function is used to delete the first document that matches conditions from the collection. It behaves like the **remove**() function but deletes **at most one document regardless of the single option.**

```
const mongoose = require('mongoose');
const User = mongoose.model('User', new mongoose.Schema({ name: String }));

async function deleteUser() {
  // connect to the database
  await mongoose.connect('mongodb://localhost:27017/test', {useNewUrlParser: true});

  let user = new User({ name: 'John' });    // create a user
  await user.save();

  // delete the user
  await User.deleteOne({ name: 'John' });

  // confirm the deletion
  const deletedUser = await User.findOne({ name: 'John' });
  console.log(deletedUser);  // should be null
}
deleteUser().catch(console.error);
```

Model.deleteOne(conditions, [callback]);

we **first** connect to the database, **then** we create a new user 'John'. We **then** delete the user 'John' from the collection. **Finally**, we try to find 'John' and it returns null, confirming the deletion.

deleteOne() will only **delete the first document that matches the condition**. If you have multiple documents that match the condition, you might want to use **deleteMany**().

populate and exec

Universidad Francisco de Vitoria
UFV Madrid

populate(): This method is used to automatically replace the specified path in the document, with document(s) from other collection(s).

We may populate a single document, multiple documents, a plain object, multiple plain objects, or all objects returned from a query.

```
Book.findOne({ title: 'Inferno' }).populate('author').exec((err, book)
=> {
 if (err) return handleError(err);
 console.log('The author is %s', book.author.name);});
```

'author' field of the book document was initially the ObjectId of the author. After we call **populate**('author'), Mongoose automatically replaces the 'author' field with the full document of the author from the 'Author' collection.

```
Book.findOne({ title: 'Inferno' }).exec((err, book) => {
 if (err) return handleError(err);
 console.log('The book is %s', book.title);
});
```

exec() method is used to execute a query. If a callback function is passed to the exec() method, Mongoose will execute the query asynchronously and pass the results to the callback.

```
User.updateOne(
 { name: 'John' },
 { $push: { comments: { content: 'A new comment' } } },
 function (err, res) {
  // Handle error or use the response here});
```

$pull operator is used to remove a comment with a certain content from the comments array of the user named 'John'.

Relationships in Mongo guarantees consistency

Universidad Francisco de Vitoria
UFV Madrid

```
{
  "_id":
ObjectId("507f1f77bcf86cd799439011"),
  "name": "Alice",
  "email": "alice@email.com"
}
```

```
{
  "_id": ObjectId("507f1f77bcf86cd799439112"),
  "title": "First Post",
  "content": "This is the content of the first post.",
  "author_id": ObjectId("507f1f77bcf86cd799439011")
}
```

With above approach to get the author's details for a particular post, you'd typically need to perform a lookup (akin to a join in SQL). This ensures data **consistency** and **avoids** data **redundancy** but we need more complex queries (requires joins/lookups). and potentially slower reads due to the need for additional lookups.

```
{
  "_id":
ObjectId("507f1f77bcf86cd799439112"),
  "title": "First Post",
  "content": "This is the content of the first
post.",
  "author": {
    "name": "Alice",
    "email": "alice@email.com"}
}
```

Each post document contains an embedded author document with the author's details. There's **no need for an additional lookup** to get author information—it's already included with each post. This gives Faster reads (no need for joins/lookups). but **Updating data in multiple places can be challenging.** and there is room for redundancy.

Relationships: References avoids data duplication

Universidad Francisco de Vitoria

UFV Madrid

So Mongo has two ways to establish relationships between data: References (**Normalization**) and Embedded Documents (**Denormalization**). Mongoose provides a way to use both approaches **with data schemas.**

```
const authorSchema = Schema({
  name: String,
  books: [{ type: Schema.Types.ObjectId, ref: 'Book' }]});

const bookSchema = Schema({
  title: String,
  author: { type: Schema.Types.ObjectId, ref: 'Author' },
  summary: String});

const Author = mongoose.model('Author', authorSchema);
const Book = mongoose.model('Book', bookSchema);
```

The **reference approach**, also known as **Normalization**, is where you reference document by storing its ObjectId in another document. This approach is akin to how foreign keys work in SQL databases.

Here you have a separate collection for each entity and you reference related documents by storing ObjectIds. This approach is similar to how relationships are handled in relational databases.

In this example, we have two entities Author and Book. An **author can have many books** and **each book has one author**. **We're referencing the related documents by storing their ObjectId.**

```
Book.find({ title: 'Inferno' })
  .populate('author')
 // This populates the 'author' field with the author's details
  .exec((err, book) => {
    if(err) console.error(err);
    console.log(book);});
```

The advantage of this approach is that it **reduces data duplication**. However, it comes at the cost of needing additional queries or a populated query (like a join operation in SQL) to get the related data.

Query in normalized and denormalized case

Universidad Francisco de Vitoria
UFV Madrid

```
db.posts.aggregate([
 {
  $lookup: {
   from: "authors", // Name of the author collection
   localField: "author_id", // Field from the posts collection
   foreignField: "_id", // Field from the authors collection
   as: "author_details" // Resulting array field
  }
 },
 {
  $project: {
   title: 1, // 1 means to include this field in the result
   author_name: "$author_details.name"
  }
 }
]);
```

In the denormalized case, querying is straightforward. However, if there's a need to update an author's name, for instance, you would have to update it in every post they authored. In the normalized case, you'd just update the authors collection and every post referencing that author would automatically be associated with the updated details.

```
db.posts.find({}, { title: 1, "author.name": 1, _id: 0 });
```

The normalized model (using references) ensures data consistency and is more storage-efficient but requires more complex queries. The denormalized model (using embedded documents) simplifies querying and can provide faster reads at the potential cost of data redundancy and increased storage. The right choice depends on the specific use case and the anticipated database operations.

Relationships: Embedded documents

Universidad Francisco de Vitoria

UFV Madrid

Embedded documents, also known as denormalization, involve nesting related data in the parent document itself. This approach can simplify querying and improve performance.

```
const mongoose = require('mongoose');
const Schema = mongoose.Schema;

const commentSchema = new Schema({
  name: String,
  content: String
});

const blogSchema = new Schema({
  title: String,
  content: String,
  comments: [commentSchema]
// Embedded comments document
});

const Blog = mongoose.model('Blog', blogSchema);
const Comment = mongoose.model('Comment',
commentSchema);
```

With this approach, you embed the related documents inside the parent document. This approach is well-suited for "contains" relationships.

In this example, we're **embedding** the **Comment documents inside** the **BlogPost** document. Each blog post contains **multiple** comments.

we embed 'Comment' documents inside the 'Blog' document. This allows us to fetch a blog post and its comments with a single database query

```
Blog
  .find({ title: 'My First Blog' })
  .exec((err, blog) => {
    if(err) console.error(err);
    console.log(blog);
  });
```

Trade-offs:. Using **References** (Normalization) can lead to **more complex queries** but it **ensures data consistency**. On the other hand, using **Embedded Documents** (Denormalization) can lead to **faster reads** but it can also lead to data redundancy.

The advantage of this approach is the ability to fetch related data in a single query, improving read performance; this comes at the cost of increased data redundancy and potential for inconsistent data.

Two-way reference

```
const authorSchema = Schema({
  name: String,
  books: [{ type: Schema.Types.ObjectId, ref: 'Book' }]
});

const bookSchema = Schema({
  title: String,
  author: { type: Schema.Types.ObjectId, ref: 'Author' },
  summary: String
});

const Author = mongoose.model('Author', authorSchema);
const Book = mongoose.model('Book', bookSchema);
```

There's a reference from each **Book** to its **Author** and from each **Author** to their **Books**.

This is **not a circular reference**, but rather **a way to establish a two-way relationship between authors and books**.

The **populate**() **method** allows you to **replace** the **ids** stored in the **author field** of **Book** documents and in the **books field** of **Author** documents with the **actual documents** from the respective collections. **Until you call populate()**, Mongoose does not fetch the related documents; it **only stores** the **ids**, which doesn't create a circular dependency problem.

Two-way reference 2 populate filling subdocument

Universidad Francisco de Vitoria
UFV Madrid

```
Book
 .findOne({ title: 'The Great Gatsby' })
 .populate('author') // This will replace the 'author' field (which is an id) with the actual author document
 .exec(function (err, book) {
  if (err) return handleError(err);
  console.log('The author is %s', book.author.name);
  // prints "The author is F. Scott Fitzgerald"
 });
```

populate('author') replaces the author's id with the actual author document from the Author collection.

```
Author
 .findOne({ name: 'F. Scott Fitzgerald' })
 .populate('books') // This will replace the 'books' field (which are ids) with the actual book documents
 .exec(function (err, author) {
  if (err) return handleError(err);
  console.log('The books are %s', author.books.map(book => book.title));
  // prints "The books are The Great Gatsby, Tender Is the Night, ..."
 });
```

populate('books') replaces the book ids in the books field with the actual book documents from the Book collection. The map() function is used to get only the book titles from the books array.

Mongoose is deprecating callbacks

Universidad Francisco de Vitoria
UFV Madrid

As of Mongoose 5.x, callbacks are no longer supported for the mongoose.connect() function. You should use Promises or async/await instead.

```
Item.insertMany(defaultItems, function (err) {  if (err) {console.log(err);  } else { console.log("yaii");}});
```

callbacks

```
Item.insertMany(defaultItems).then=> {
   console.log("yaii");
 }).catch((err) => {console.log(err);});
```

Item.insertMany(defaultItems) initiates the operation to insert the documents. This returns a **Promise**.

.then() is called when the Promise is resolved

.catch() is called when the Promise is rejected

```
async function connectDB() { // Using async/await
  try {
    await mongoose.connect('mongodb://localhost:27017/myapp', { useNewUrlParser: true, useUnifiedTopology: true });
    console.log('Database connection successful');
  } catch (err) {
    console.error('Database connection error: ', err); }
}
connectDB();
```

mongoose data analysis

Universidad Francisco de Vitoria
UFV Madrid

```
> show databases
admin       0.000GB
blog        0.000GB
config      0.000GB
local       0.000GB
todolistDB  0.000GB
> use todolistDB
switched to db todolistDB
> show tables
items
todolists
> db.todolists.find()
> db.items.find()
{ "_id" : ObjectId("6520a6765bf432941ba30d9f"), "name" : "Breathe!", "__v" : 0 }
{ "_id" : ObjectId("6520a6765bf432941ba30da1"), "name" : "Be positive!", "__v" : 0 }
{ "_id" : ObjectId("6520a6765bf432941ba30da0"), "name" : "Smile!", "__v" : 0 }
{ "_id" : ObjectId("6520ac145bf432941ba30dd7"), "name" : "", "__v" : 0 }
```

The **primary key** for a MongoDB document, automatically generated by MongoDB when a new document is created and is unique to the document. The **_id** field is an **ObjectId** which is a **12-byte BSON type**, designed to have a **high probability of being unique when generated.**

version key (named **__v** by default in Mongoose). It's used internally by Mongoose to manage document **versioning and to handle document updates correctly**, especially when dealing with concurrent operations that could lead to conflicts.

The rest of the fields in a document represent the data that you have stored. These fields are defined according to the schema you create in Mongoose. The schema defines the structure of the document, including types of data, validation, and default values.

example composition where I used embedded for simplicity

Universidad Francisco de Vitoria
UFV Madrid

```
const itemsSchema = {
  name: String};
```

```
const listSchema = {
  name: String,
  items: [itemsSchema]
};
```

```
const item1 = new Item({ name: "Welcome to your todolist!" });
const item2 = new Item({ name: "Hit the + button to add a new item." });
const item3 = new Item({ name: "<-- Hit this to delete an item." });
const defaultItems = [item1, item2, item3];
```

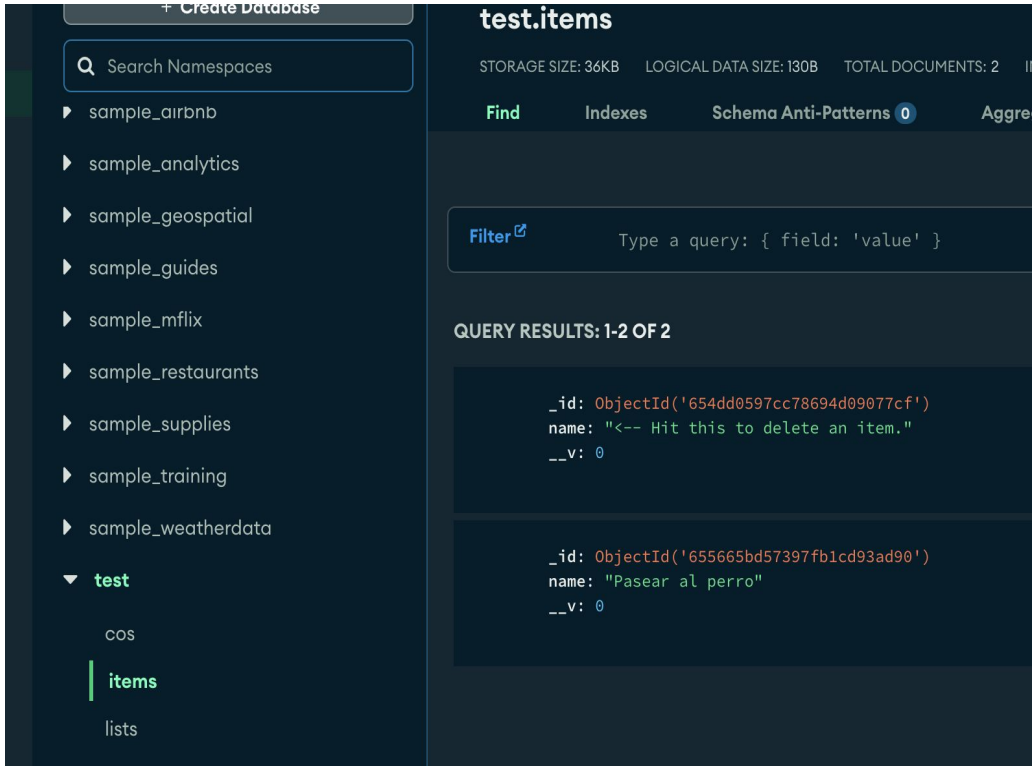
```
const Item = mongoose.model("Item", itemsSchema);
```

```
const List = mongoose.model("List", listSchema);
```

1. In listSchema, the items field is not just a reference to itemsSchema; it's an array that **directly contains objects conforming to itemsSchema.**
2. This means that **each document in the collection corresponding to listSchema will contain an embedded array of item objects**. Each item object in this array will follow the structure defined in itemsSchema.
3. There is **no separate collection for items**. Instead, items are embedded directly within each list document.

The approach used here is known as **embedding** or **denormalization**. Instead of storing item documents in a separate collection and referencing them in the list documents (which would be a **normalized approach**), you store the entire item object within the list document. This results in a **duplication** of **item data** if the same item is used in multiple lists but it **simplifies queries**, as you don't need to perform a **separate lookup** or **join** to retrieve item details for a list.

main Today list uses just items



When an item is added to the "Today" list, the application directly creates and saves a **new Item document in the MongoDB collection** corresponding to Item model. The new item's name is obtained from req.body.newItem. A new Item document is created with this name and then saved to the MongoDB database.

Upon successful saving, the application redirects the user to the home page ("/") to display the "Today" list.

The primary difference lies in data handling for the default list versus custom lists. Default "Today" list operates directly with the Item collection, adding each new item as a separate document. In contrast, custom lists maintain an array of items within each list document, embedding the item data directly in the list.

This approach reflects a flexible use of MongoDB, utilizing both standalone documents for the "Today" list and embedded documents for custom lists, providing efficient data access and manipulation depending on the list context.

Rest of lists are in a collection of lists



When a new list is created, you can directly **embed item objects** within the list document. When the list is queried, MongoDB will return the entire list document, including the embedded items.

- The choice between embedding (denormalization) and referencing (normalization) depends on the specific requirements of your application, such as the query patterns, data size, and update frequency.
- Embedded documents offer better performance for read operations but can make updates more complex, especially if the same embedded data is duplicated across multiple documents.

Privacity, do not expose keys, passwords, etc…

MongoDB.

We have become aware that active database credentials for your MongoDB Atlas cluster are publicly accessible on GitHub at https://github.com/jjimnz11/express-mongo-hands-on/blob/a563f5e892b7ea8c3ea393422056784 61d2fe63d/awcodeAtlas.js for your MongoDB Atlas project https://cloud.mongodb.com/v2/6347795339713 64cd0ba42e6#/overview. This may have been intentional, however we would like to ensure that you are fully aware of this potential security concern.

Please note that these credentials are used to access the databases associated with the project listed above. These are not the credentials that you use to access your Atlas account at cloud.mongodb.com.

Phase 1: Immediate Security Fix

Universidad Francisco de Vitoria
UFV Madrid

1. **Log in to MongoDB Atlas.**
2. Navigate to **Security** $\rightarrow$ **Database Access** on the left sidebar.
3. Find your user (e.g., admin) and click **Edit**.
4. **Change the Password:** Generate a new, strong password.
5. **Save Changes.**
    - *Result:* The URL currently exposed on GitHub is now useless to hackers.

Phase 2: Local Development Setup

Universidad Francisco de Vitoria
UFV Madrid

This is how you configure the project to keep secrets safe on your computer.

1. Install the `dotenv` package Open your terminal in the project folder → npm install dotenv

2. Create the `.env` file Create a file named `.env` in the root of your project. Add your connection string here:

DB_URI=mongodb+srv://your_user:NEW_PASSWORD@cluster0.mongodb.net/your_db_name
PORT=3000

(*Note: Do not use quotes or spaces around the = sign).*

3. Create the `.gitignore` file (*This is the most critical step as tells Git: "Upload everything, but ignore my secret `.env` file")* and add:

node_modules
.env

4. Create a `.env.example` file (Best Practice for teaching) Create a file named `.env.example`. This *will* be uploaded to GitHub to show others what variables they need.

DB_URI=insert_your_connection_string_here
PORT=3000

Update your Code

Universidad Francisco de Vitoria
UFV Madrid

Modify your main file (e.g., app.js or index.js) to load the variables:

```
require('dotenv').config(); // Loads the variables from .env

const mongoose = require('mongoose');
const dbUri = process.env.DB_URI; // Access the variable

mongoose.connect(dbUri)
  .then(() => console.log('Connected to MongoDB safely!'))
  .catch(err => console.log(err));
```

Phase 3: Deploying to the Cloud (Render, Vercel, etc.)

Since the .env file is ignored by Git, it won't exist in the cloud. You must inject the variables manually.

1. Go to your **Render Dashboard** (or your chosen cloud provider).
2. Select your project.
3. Find the **"Environment"** or **"Environment Variables"** tab.
4. Click **"Add Environment Variable"**.
5. **Key:** DB_URI
6. **Value:** mongodb+srv://your_user:NEW_PASSWORD@cluster0.mongodb.net/your_db_name
7. Save. The application will restart and connect successfully.

Act with repo without any password visible

Universidad Francisco de Vitoria
UFV Madrid

1. **Fork & Clone:** They clone my repository to your local machine.
2. **Install Dependencies** → run **`npm install`**.
3. **Create Secrets:** Create your own **`.env`** file (copying the structure from `.env.example`) if exists
4. **Add Credentials:**
   - *Scenario A:* Using your own MongoDB Atlas URL (Recommended for learning).
   - *Scenario B:* Use my connection string that I may share privately (via Slack/Discord/LMS) not in a public repo, in that case you can paste it in your file **`.env`**
5. **Run:** Then run `npm start`, and it works securely.